

1. 건물 정보 조회 API

위치

파일: `src/main/java/com/memeki/reviewhome/postAddress/service/PostAddressService.java`

핵심 코드

1.1 API 선택 로직 (92-99줄)

```
if (postAddress == null) {
    // is-open-api-temp 값에 따라 서울시 API 또는 기존 API 사용
    if (isOpenApiTemp && addressInfo.getSigunguCd().startsWith("11")) {
        return searchAddressFromSeoul(addressInfo);
    } else {
        return searchAddress(addressInfo);
    }
}
```

설명:

- DB에 주소가 없을 때 API 호출
- `is-open-api-temp=true`이고 시군구코드가 "11"로 시작(서울)이면 서울시 임시 API 사용
- 그 외에는 공공 데이터 포털 API 사용

1.2 공공 데이터 포털 API 호출 (156-177줄)

```
GetBrTitleInfoResponseDto result = webClient.get()
    .uri(
        uriBuilder -> {
            UriBuilder builder =
uriBuilder.path("/1613000/BldRgstHubService/getBrTitleInfo")
                .queryParams("sigunguCd", addressInfo.getSigunguCd())
                .queryParams("bjdongCd", addressInfo.getBjdongCd())
                .queryParams("bun", addressInfo.getBun())
                .queryParams("numOfRows", numOfRows)
                .queryParams("pageNo", pageNo.get());

                // ji가 null이 아닌 경우에만 쿼리 파라미터 추가
                if (addressInfo.getJi() != null) {
                    builder.queryParam("ji", addressInfo.getJi());
                }
                return builder.build();
            })
    .retrieve()
    .bodyToMono(GetBrTitleInfoResponseDto.class)
    .doOnError(WebClientResponseException.class, ex -> {
        // 응답값 확인
    })
```

```

        System.out.println("GetBrTitleInfoResponseDto ---> " +
ex.getResponseBodyAsString());
    }).block();

```

설명:

- **API 엔드포인트:** `/1613000/BldRgstHubService/getBrTitleInfo`
- **용도:** 건물 등기 정보 조회
- **파라미터:** 시군구코드, 법정동코드, 번, 지, 페이지 번호, 페이지당 결과 수
- **페이징:** `pageNo`와 `numOfRows`로 여러 페이지 조회 가능
- **에러 처리:** `doOnError`로 에러 응답 로깅

1.3 서울시 임시 API 호출 (447-475줄)

```

public SearchAddressDto.PostResponse
searchAddressFromSeoul(SearchAddressDto.Request addressInfo) throws Exception {
    try {
        SearchAddressDto.PostResponse response = new
SearchAddressDto.PostResponse();
        List<GetBrTitleInfoResponseDto.ItemDto> allItems = new ArrayList<>();
        List<String> dongNmList = new ArrayList<>();

        // 서울시 API는 시군구코드가 11로 시작하는 경우만 처리
        if (!addressInfo.getSigunguCd().startsWith("11")) {
            throw new DefaultException(ErrorCode.NOT_FOUND);
        }

        // 서울시 API 호출 (WebClient 없이 직접 URL 생성)
        String apiUrl = String.format(
            "http://openapi.seoul.go.kr:8088/%s/json/vBigDjrTitle/1/1000",
            tempOpenApiKey
        );

        WebClient seoulWebClient = WebClient.builder()
            .baseUrl(apiUrl)
            .codecs(configurer ->
configurer.defaultCodecs().maxInMemorySize(10 * 1024 * 1024)) // 10MB로 증가
            .build();

        SeoulBuildingResponseDto result = seoulWebClient.get()
            .retrieve()
            .bodyToMono(SeoulBuildingResponseDto.class)
            .doOnError(WebClientResponseException.class, ex -> {
                System.out.println("SeoulBuildingResponseDto Error ---> " +
ex.getResponseBodyAsString());
            })
            .block();
    }
}

```

설명:

- **API 엔드포인트:** `http://openapi.seoul.go.kr:8088/{API_KEY}/json/vBigDjrTitle/1/1000`
- **용도:** 데이터센터 화재로 인한 공공 API 장애 시 서울 지역 건물 정보 조회
- **특징:**
 - 서울 지역(시군구코드 "11"로 시작)만 지원
 - 최대 1000건 한 번에 조회
 - 10MB 버퍼 크기로 대용량 응답 처리
- **활성화 조건:** `is-open-api-temp=true` 설정 필요

2. 상권 정보 조회 API

위치

파일: `src/main/java/com/memeki/reviewhome/commercial/service/CommercialService.java`

핵심 코드

2.1 상권 정보 조회 메인 로직 (54-106줄)

```
@Transactional
public void fetchAndSaveCommercialInfo(String postAddressInfoUuid, double pointX,
double pointY) {
    // 상권 API가 비활성화된 경우 스킵
    if (!commercialApiEnabled) {
        log.info("상권 조회 API 비활성화 상태 - 조회 스킵 - UUID: {}",
postAddressInfoUuid);
        log.info("활성화 방법: application.properties에서 commercial-api-
enabled=true로 설정");
        return;
    }

    // 임시 API 모드일 때는 상권 정보 조회를 스킵
    if (isOpenApiTemp) {
        log.info("임시 API 모드 - 상권 정보 조회 스킵 - UUID: {}",
postAddressInfoUuid);
        return;
    }

    try {
        log.info("상권 정보 조회 시작 - UUID: {}, X: {}, Y: {}",
postAddressInfoUuid, pointX, pointY);

        // 기존 데이터가 있으면 삭제
        if
(commercialInfoRepository.existsByPostAddressInfoUuid(postAddressInfoUuid)) {
commercialInfoRepository.deleteByPostAddressInfoUuid(postAddressInfoUuid);
            log.info("기존 상권 정보 삭제 완료 - UUID: {}", postAddressInfoUuid);
        }

        // 외부 API 호출
```

```

        CommercialApiResponseDto response = callCommercialApi(pointX, pointY);

        if (response != null && response.getBody() != null &&
response.getBody().getItems() != null) {
            List<CommercialInfo> commercialInfoList = new ArrayList<>();

            for (Object item : response.getBody().getItems()) {
                try {
                    // Object를 CommercialItemDto로 변환
                    CommercialItemDto commercialItemDto =
objectMapper.convertValue(item, CommercialItemDto.class);
                    CommercialInfo commercialInfo =
convertToEntity(commercialItemDto, postAddressInfoUuid);
                    commercialInfoList.add(commercialInfo);
                } catch (Exception e) {
                    log.warn("상권 정보 항목 변환 실패 - UUID: {}, 항목: {}",
postAddressInfoUuid, item, e);
                }
            }

            // 일괄 저장
            commercialInfoRepository.saveAll(commercialInfoList);
            log.info("상권 정보 저장 완료 - UUID: {}, 저장된 개수: {}",
postAddressInfoUuid, commercialInfoList.size());
        } else {
            log.warn("상권 정보 응답이 비어있음 - UUID: {}", postAddressInfoUuid);
        }

    } catch (Exception e) {
        log.error("상권 정보 조회 및 저장 중 오류 발생 - UUID: {}",
postAddressInfoUuid, e);
        throw new RuntimeException("상권 정보 처리 중 오류가 발생했습니다.", e);
    }
}

```

설명:

- **용도:** 특정 좌표 기준 반경 2000m 내 상권 정보 조회 및 저장
- **비활성화 조건:**
 - `commercial-api-enabled=false` (현재 상태)
 - `is-open-api-temp=true` (서울시 임시 API 사용 시)
- **처리 흐름:** API 호출 → DTO 변환 → DB 저장

2.2 상권 API 호출 (111-135줄)

```

private CommercialApiResponseDto callCommercialApi(double cx, double cy) {
    try {
        WebClient webClient = webClientBuilder.build();

        return webClient.get()
            .uri(uriBuilder -> uriBuilder

```

```

        .scheme("https")
        .host("apis.data.go.kr")
        .path("/B553077/api/open/sdsc2/storeListInRadius")
        .queryParams("ServiceKey", openApiKey)
        .queryParams("pageNo", 1)
        .queryParams("numOfRows", 1000)
        .queryParams("radius", MAX_RADIUS)
        .queryParams("cx", cx)
        .queryParams("cy", cy)
        .build())
    .retrieve()
    .bodyToMono(CommercialApiResponseDto.class)
    .block();

} catch (Exception e) {
    log.error("상권 API 호출 중 오류 발생 - X: {}, Y: {}", cx, cy, e);
    return null;
}
}

```

설명:

- **API 엔드포인트:** <https://apis.data.go.kr/B553077/api/open/sdsc2/storeListInRadius>
- **용도:** 좌표 기준 반경 내 상권 정보 조회
- **파라미터:**
 - **cx, cy:** 중심 좌표 (경도, 위도)
 - **radius:** 반경 (2000m 고정)
 - **pageNo:** 페이지 번호 (1)
 - **numOfRows:** 페이지당 결과 수 (1000)

3. 주소 좌표 변환 API

위치

파일: [src/main/java/com/memeki/reviewhome/geo/service/GeoService.java](#)

핵심 코드

3.1 VWorld API WebClient 설정 (34-36줄)

```

public GeoService(WebClient.Builder webClientBuilder, GeoLocationRepository
geoLocationRepository) {
    this.webClient = webClientBuilder.baseUrl("https://api.vworld.kr").build();
    this.geoLocationRepository = geoLocationRepository;
}

```

설명:

- **Base URL:** <https://api.vworld.kr>
- **용도:** 주소를 좌표로 변환하는 VWorld API 호출

3.2 주소 좌표 변환 API 호출 (48-126줄)

```

public AddressToPointsResponseDto addressToPoints(String address, String uuid, int
post_address_id,
    @Nullable String dongNm)
    throws DefaultException {
    AtomicInteger _page = new AtomicInteger(1); // AtomicInteger로 변경
    int _size = 10; // 페이지당 검색 결과 수
    boolean found = false;
    VWorldApiResponseResponseDto.PointDTO point = null;

    while (!found) {
        // VWorld API 호출
        VWorldApiResponseResponseDto vworldApiResponseResponseDto = this.webClient.get()
            .uri(uriBuilder -> uriBuilder
                .path("/req/search")
                .queryParams("key", vworldApiKey)
                .queryParams("service", "search")
                .queryParams("request", "search")
                .queryParams("type", "address")
                .queryParams("category", "road")
                .queryParams("query", address)
                .queryParams("page", _page.get())
                .queryParams("size", _size)
                .build())
            .retrieve()
            .bodyToMono(VWorldApiResponseResponseDto.class)
            .block(); // 동기식 호출
        if (vworldApiResponseResponseDto.getResponse().getResult() != null) {
            // 응답에서 items를 추출합니다.
            List<VWorldApiResponseResponseDto.ItemDTO> items =
vworldApiResponseResponseDto.getResponse().getResult().getItems();

            // items가 비어있지 않은 경우 동 이름을 검사합니다.
            for (VWorldApiResponseResponseDto.ItemDTO item : items) {
                if (dongNm == null) {
                    String targetA = item.getAddress().getRoad().replaceAll("\\
(.*?\\)", "").trim();
                    String targetB = address.replaceAll("\\(.*?\\)", "").trim();
                    if (targetA.equals(targetB)) {
                        point = item.getPoint();
                        found = true;
                        break;
                    }
                }
            }
            String dong_road = item.getAddress().getRoad() != null
                ?
Integer.toString(extractDongNumber(item.getAddress().getRoad()))
                : "";

```

```

        String dong_bldnm = item.getAddress().getBldnm() != null
            ?
Integer.toString(extractDongNumber(item.getAddress().getBldnm()))
            : "";
        String dong_bldnmdc = item.getAddress().getBldnmdc() != null
            ?
Integer.toString(extractDongNumber(item.getAddress().getBldnmdc()))
            : "";

        if (dongNm != null && dongNm.equals(dong_bldnmdc)) {
            point = item.getPoint();
            found = true;
            break;
        } else if (dongNm != null && dongNm.equals(dong_bldnm)) {
            point = item.getPoint();
            found = true;
            break;
        } else if (dongNm != null && dongNm.equals(dong_road)) {
            point = item.getPoint();
            found = true;
            break;
        }
    }

    // 다음 페이지로 이동
    if (!found) {
        // 만약 다음 페이지가 없으면 반복 종료
        if
(Integer.parseInt(vworldApiResponseDto.getResponse().getRecord().getTotal()) <=
_page.get()
        * _size) {
            break;
        }
        _page.incrementAndGet();
        if (items.isEmpty()) { // 더 이상 결과가 없으면 반복 종료
            break;
        }
    }
}

if (point != null) {
    GeoLocation geoLocation = new GeoLocation();
    geoLocation.setPointX(point.getX());
    geoLocation.setPointY(point.getY());
    geoLocation.setPostAddressInfoUuid(uuid);
    geoLocationRepository.save(geoLocation);

    AddressToPointsResponseDto addressToPointsResponseDto = new
AddressToPointsResponseDto();
    addressToPointsResponseDto.setStatusCode(200);
    addressToPointsResponseDto.setPoint_x(point.getX());
    addressToPointsResponseDto.setPoint_y(point.getY());
    return addressToPointsResponseDto;
} else {

```

```
        throw new DefaultException(ErrorCode.NOT_FOUND);  
    }  
}
```

설명:

- **API 엔드포인트:** <https://api.vworld.kr/req/search>
 - **용도:** 도로명 주소를 좌표(경도, 위도)로 변환
 - **파라미터:**
 - **key:** VWorld API 키
 - **service:** "search"
 - **request:** "search"
 - **type:** "address"
 - **category:** "road" (도로명 주소)
 - **query:** 검색할 주소
 - **page:** 페이지 번호
 - **size:** 페이지당 결과 수 (10)
 - **페이징:** 여러 페이지를 순회하며 정확한 주소 매칭
 - **동명 매칭:** **dongNm**이 있으면 건물명, 도로명에서 동명 추출하여 매칭
 - **결과 저장:** 좌표를 **GeoLocation** 테이블에 저장
-